

# lab2 配置Web服务器，分析HTTP交互过程

才子桓 2212700 密码科学与技术

## 实验要求

- (1) 搭建 Web 服务器（自由选择系统），并制作简单的 Web 页面，包含简单文本信息（至少包含专业、学号、姓名）、自己的 LOGO、自我介绍的音频信息。
- (2) 通过浏览器获取自己编写的 Web 页面，使用 Wireshark 捕获浏览器与 Web 服务器的交互过程，使用 Wireshark 过滤器使其只显示 HTTP 协议。
- (3) 现场演示。
- (4) 提交 HTML 文档、Wireshark 捕获文件和实验报告，对 HTTP 交互过程进行详细说明。

**注：**页面不要太复杂，包含所要求的基本信息即可。使用 HTTP，不要使用 HTTPS。

## 实验说明

本次实验为自行配置 Web 服务器，并分析访问该 Web 时的交互过程。这里我选择将服务器建立在 Ubuntu 系统上，利用 Python 语言中 `python3 -m http.server 8000` 指令开启 8000 端口的访问权限。Web 我选择使用 html 语言，建立一个 `index.html` 作为访问界面。

在 Windows 主机上访问 Ubuntu IP:8000 进行访问 Web。在 Wireshark 上选择虚拟机的网卡 `vmware Network Adapter VMnet8`，添加筛选条件进行抓包。

## 实验环境

**Web环境：** Ubuntu 20.04 LTS

**抓包环境：** Windows 11

Web语言：html

编辑器：WebStorm 2024.1

抓包工具：Wireshark

音频合成语言：Python 3.12

音频合成工具包：gtts

音频合成编辑器：PyCharm Community Edition 2024.2.3

## 实验过程

1. 合成个人介绍音频：

由于某视频剪辑软件上音色需要收费，所以这里我选择使用 Python 合成音频，需要下载 gtts 库，代码如下。

```

import os
from gtts import gTTS

# 禁用代理
os.environ["HTTP_PROXY"] = ""
os.environ["HTTPS_PROXY"] = ""

text = "你好，我叫才子桓，是一名22级网络空间安全学院的本科生。这是我的计算机网络的实验二的web界面。"
language = 'zh'
tts = gTTS(text=text, lang=language)
tts.save("output.mp3")
print("success")

```

生成成功。



2. 编辑 index.html:

代码如下。

```

<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hi</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 20px;
    }
    header {
      background-color: #4CAF50;
      color: white;
      padding: 10px 0;
      text-align: center;
    }
    main {
      background-color: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      max-width: 800px;
      margin: 20px auto;
    }
  </style>
</head>
<body>
  <div>
    <h1>Hi</h1>
  </div>
</body>
</html>

```

```
    }
    img {
        max-width: 100%;
        height: auto;
        border-radius: 8px;
    }
    audio {
        display: block;
        margin: 20px 0;
    }
    a {
        color: #4CAF50;
        text-decoration: none;
    }
    a:hover {
        text-decoration: underline;
    }
</style>
</head>
<body>

<header>
    <h1>welcome to my web!</h1>
</header>

<main>
    <h2>Abort Me</h2>
    
    <p>Name: 才子桓</p>
    <p>My student ID: 2212700</p>
    <p>College: 网络空间安全学院</p>
    <p>Major: 密码科学与技术</p>
    <p>Email: caizihuan0904@126.com</p>

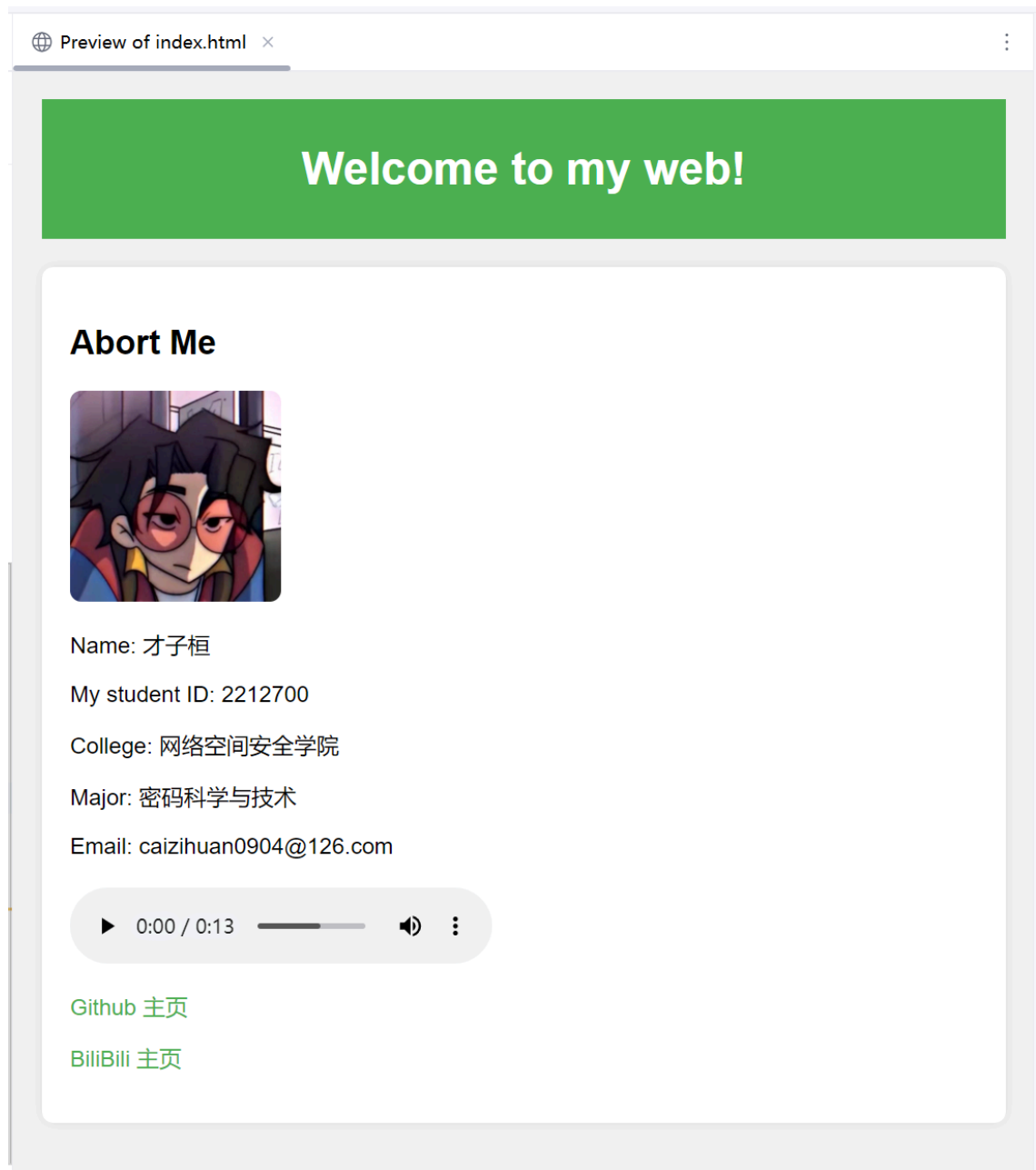
    <audio controls>
        <source src="output.mp3" type="audio/mpeg">
    </audio>

    <h3></h3>
    <p><a href="https://github.com/Scnge" target="_blank">Github 主页</a>
</p>
    <p><a href="https://space.bilibili.com/413721864?spm_id_from=333.1007.0.0" target="_blank">Bilibili 主页</a></p>

</main>

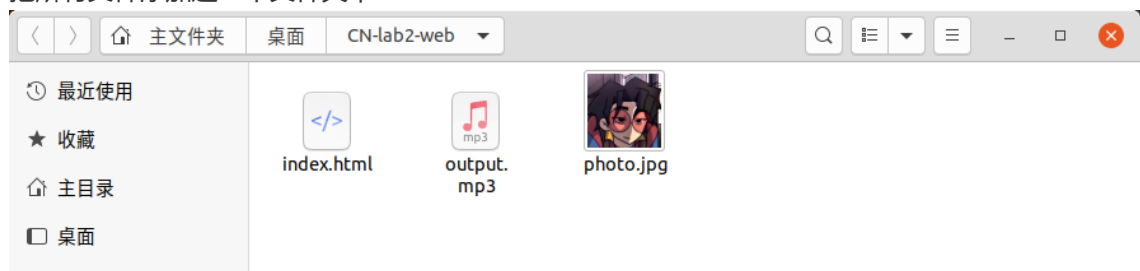
</body>
</html>
```

运行界面：



3. 在 Ubuntu 上启动访问端口：

把所有文件添加进一个文件夹中：



在这个文件夹中打开终端，输入：

```
python3 -m http.server 8000
```

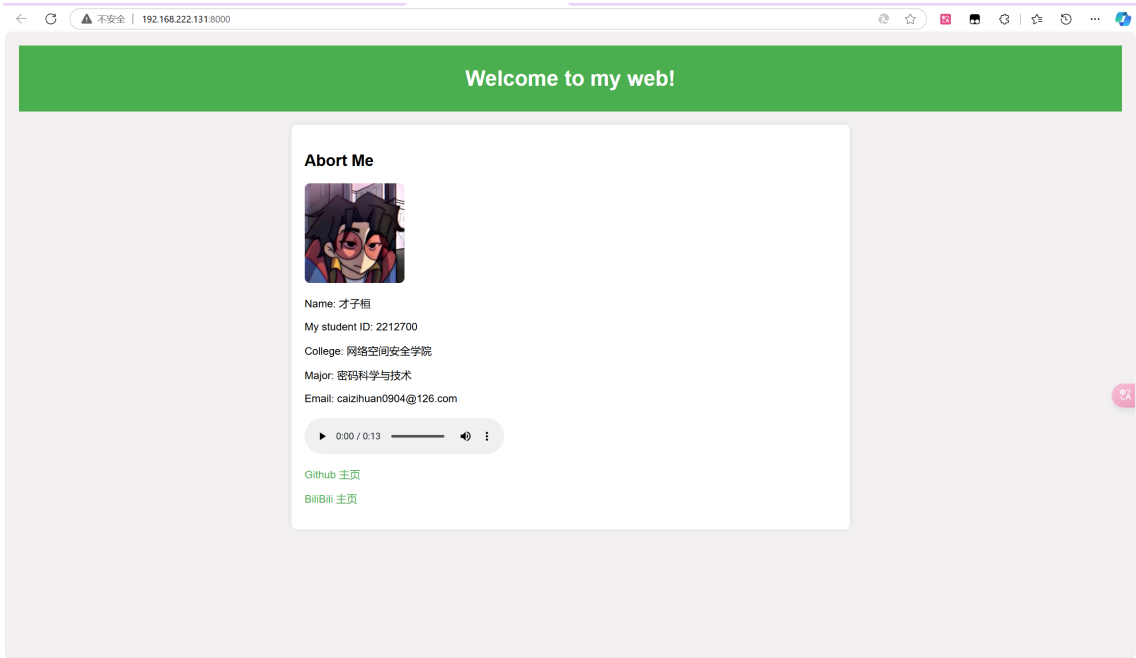
这样我们就在 Ubuntu 上部署了我们的 Web 项目。

查看 Ubuntu 虚拟机 IP 地址：

```
hostname -I
```

```
cai@cai-virtual-machine:~$ hostname -I
192.168.222.131
cai@cai-virtual-machine:~$
```

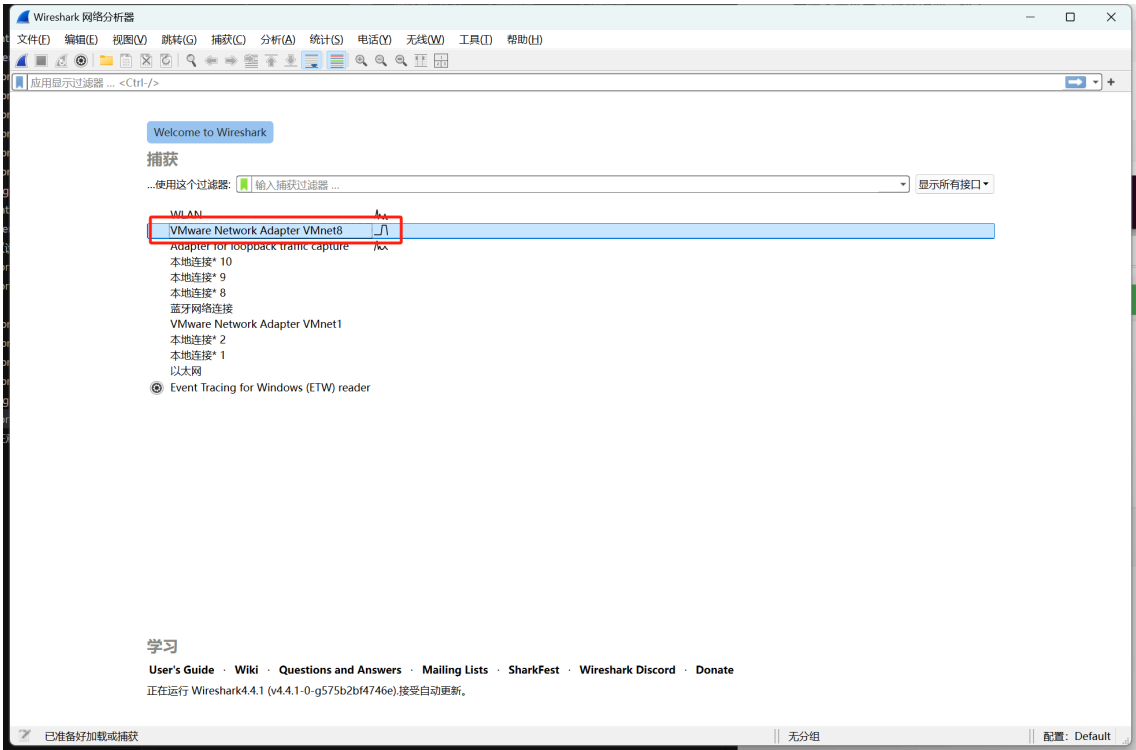
我们尝试在 Windows 主机上查看 Web 界面：



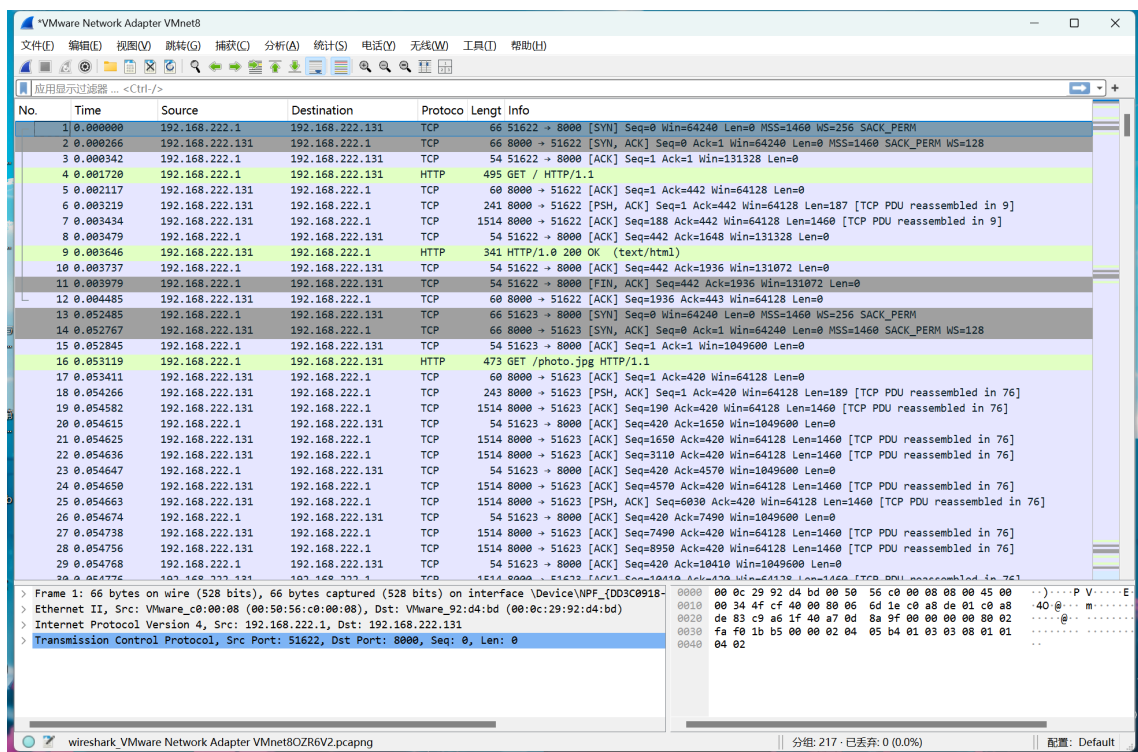
访问成功，部署完成。

4. 利用 Wireshark 抓包：

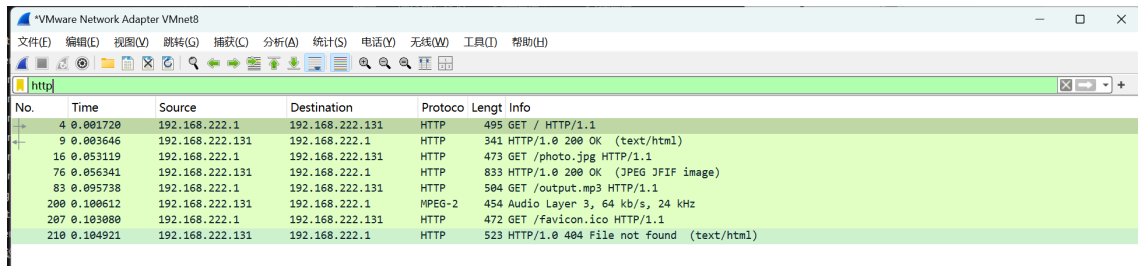
打开 Wireshark，由于我们 Web 部署在虚拟机上，所以我们选择捕获 VMware Network Adapter VMnet8 上的数据包。



捕获详情：



添加筛选条件 http，筛出 http 协议的数据包。



## 实验分析

### 1. TCP 三次握手协议分析：

TCP/IP协议中，TCP协议提供可靠的连接服务，采用三次握手建立一个连接。

(1) **第一次握手：** 建立连接时，客户端A发送SYN包 (SYN=j) 到服务器B，并进入SYN\_SEND状态，等待服务器B确认。

(2) **第二次握手：** 服务器B收到SYN包，必须确认客户A的SYN (ACK=j+1)，同时自己也发送一个SYN包 (SYN=k)，即SYN+ACK包，此时服务器B进入SYN\_RECV状态。

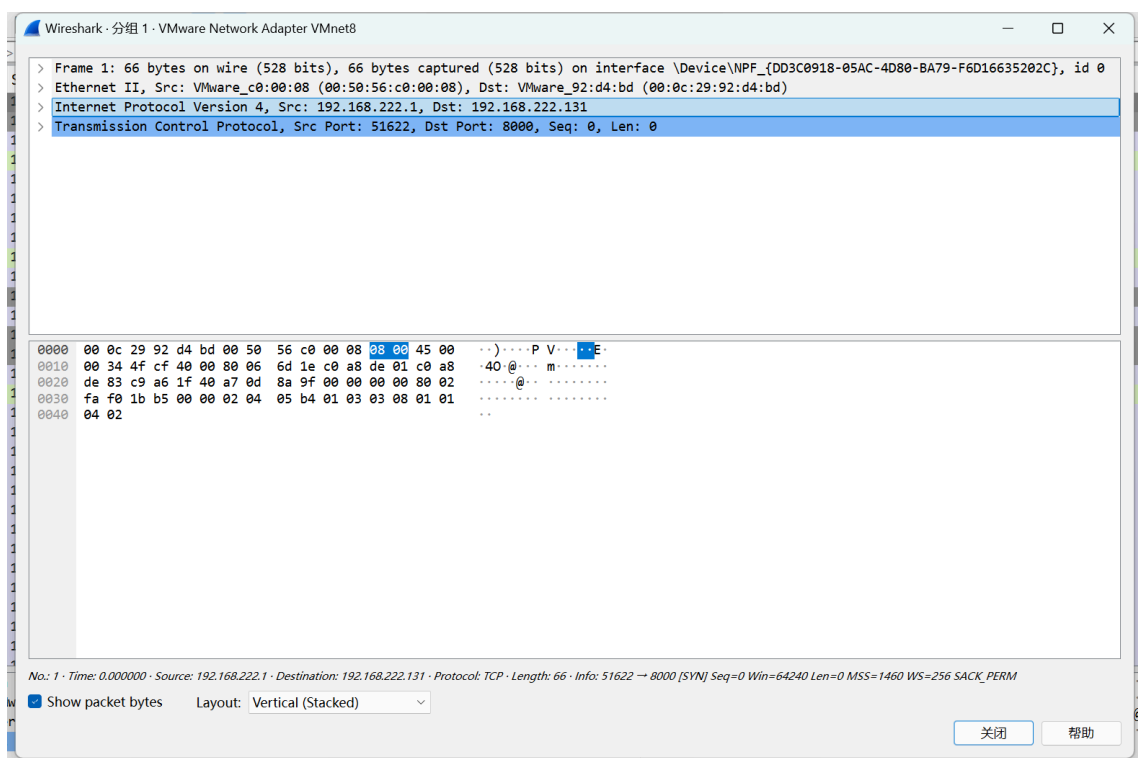
(3) **第三次握手：** 客户端A收到服务器B的SYN + ACK包，向服务器B发送确认包ACK (ACK=k+1)，此包发送完毕，客户端A和服务器B进入ESTABLISHED状态，完成三次握手。

完成三次握手，客户端与服务器开始传送数据。

| No. | Time     | Source          | Destination     | Protocol | Length | Info                                                                                    |
|-----|----------|-----------------|-----------------|----------|--------|-----------------------------------------------------------------------------------------|
| 1   | 0.000000 | 192.168.222.1   | 192.168.222.131 | TCP      | 66     | 51622 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM                      |
| 2   | 0.000266 | 192.168.222.131 | 192.168.222.1   | TCP      | 66     | 8000 → 51622 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128           |
| 3   | 0.000342 | 192.168.222.1   | 192.168.222.131 | TCP      | 54     | 51622 → 8000 [ACK] Seq=1 Ack=1 Win=131328 Len=0                                         |
| 4   | 0.001720 | 192.168.222.1   | 192.168.222.131 | HTTP     | 495    | GET / HTTP/1.1                                                                          |
| 5   | 0.002117 | 192.168.222.131 | 192.168.222.1   | TCP      | 60     | 8000 → 51622 [ACK] Seq=1 Ack=442 Win=64128 Len=0                                        |
| 6   | 0.003219 | 192.168.222.131 | 192.168.222.1   | TCP      | 241    | 8000 → 51622 [PSH, ACK] Seq=1 Ack=442 Win=64128 Len=187 [TCP PDU reassembled in 9]      |
| 7   | 0.003434 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51622 [ACK] Seq=188 Ack=442 Win=64128 Len=1460 [TCP PDU reassembled in 9]        |
| 8   | 0.003479 | 192.168.222.131 | 192.168.222.131 | TCP      | 54     | 51622 → 8000 [ACK] Seq=442 Ack=1648 Win=131328 Len=0                                    |
| 9   | 0.003646 | 192.168.222.131 | 192.168.222.1   | HTTP     | 341    | HTTP/1.0 200 OK (text/html)                                                             |
| 10  | 0.003737 | 192.168.222.1   | 192.168.222.131 | TCP      | 54     | 51622 → 8000 [ACK] Seq=442 Ack=1936 Win=131072 Len=0                                    |
| 11  | 0.003979 | 192.168.222.1   | 192.168.222.131 | TCP      | 54     | 51622 → 8000 [FIN, ACK] Seq=442 Ack=1936 Win=131072 Len=0                               |
| 12  | 0.004485 | 192.168.222.131 | 192.168.222.1   | TCP      | 60     | 8000 → 51622 [ACK] Seq=1936 Ack=443 Win=64128 Len=0                                     |
| 13  | 0.052485 | 192.168.222.1   | 192.168.222.131 | TCP      | 66     | 51623 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM                      |
| 14  | 0.052767 | 192.168.222.131 | 192.168.222.1   | TCP      | 66     | 8000 → 51623 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128           |
| 15  | 0.052845 | 192.168.222.1   | 192.168.222.131 | TCP      | 54     | 51623 → 8000 [ACK] Seq=1 Ack=1 Win=1049600 Len=0                                        |
| 16  | 0.053119 | 192.168.222.1   | 192.168.222.131 | HTTP     | 473    | GET /photo.jpg HTTP/1.1                                                                 |
| 17  | 0.053411 | 192.168.222.131 | 192.168.222.1   | TCP      | 60     | 8000 → 51623 [ACK] Seq=1 Ack=420 Win=64128 Len=0                                        |
| 18  | 0.054266 | 192.168.222.131 | 192.168.222.1   | TCP      | 243    | 8000 → 51623 [PSH, ACK] Seq=1 Ack=420 Win=64128 Len=189 [TCP PDU reassembled in 76]     |
| 19  | 0.054582 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=190 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]       |
| 20  | 0.054615 | 192.168.222.131 | 192.168.222.131 | TCP      | 54     | 51623 → 8000 [ACK] Seq=420 Ack=1650 Win=1049600 Len=0                                   |
| 21  | 0.054625 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=1650 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]      |
| 22  | 0.054636 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=3110 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]      |
| 23  | 0.054647 | 192.168.222.131 | 192.168.222.131 | TCP      | 54     | 51623 → 8000 [ACK] Seq=420 Ack=4570 Win=1049600 Len=0                                   |
| 24  | 0.054650 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=4570 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]      |
| 25  | 0.054663 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [PSH, ACK] Seq=6030 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76] |
| 26  | 0.054674 | 192.168.222.131 | 192.168.222.131 | TCP      | 54     | 51623 → 8000 [ACK] Seq=420 Ack=7490 Win=1049600 Len=0                                   |
| 27  | 0.054738 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=7490 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]      |
| 28  | 0.054756 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=8950 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]      |
| 29  | 0.054768 | 192.168.222.131 | 192.168.222.131 | TCP      | 54     | 51623 → 8000 [ACK] Seq=420 Ack=10410 Win=1049600 Len=0                                  |
| 30  | 0.054796 | 192.168.222.131 | 192.168.222.1   | TCP      | 1514   | 8000 → 51623 [ACK] Seq=10410 Ack=420 Win=64128 Len=1460 [TCP PDU reassembled in 76]     |

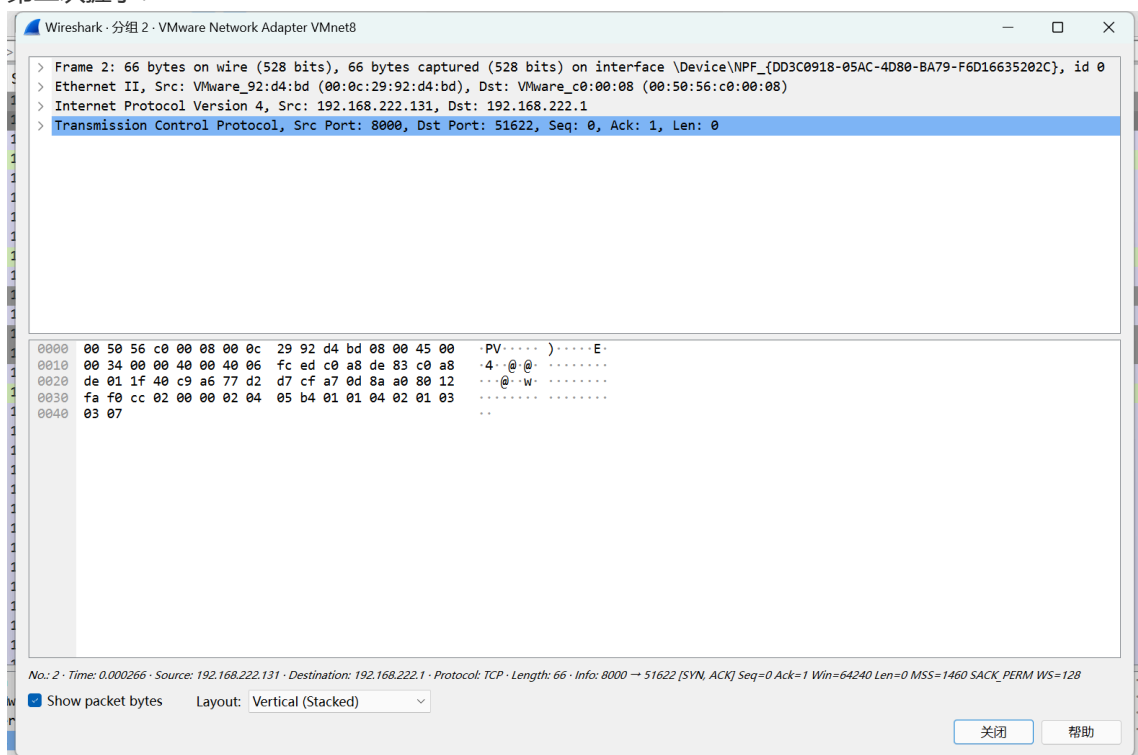
下面对三次握手的数据包进行详细分析：

第一次握手：



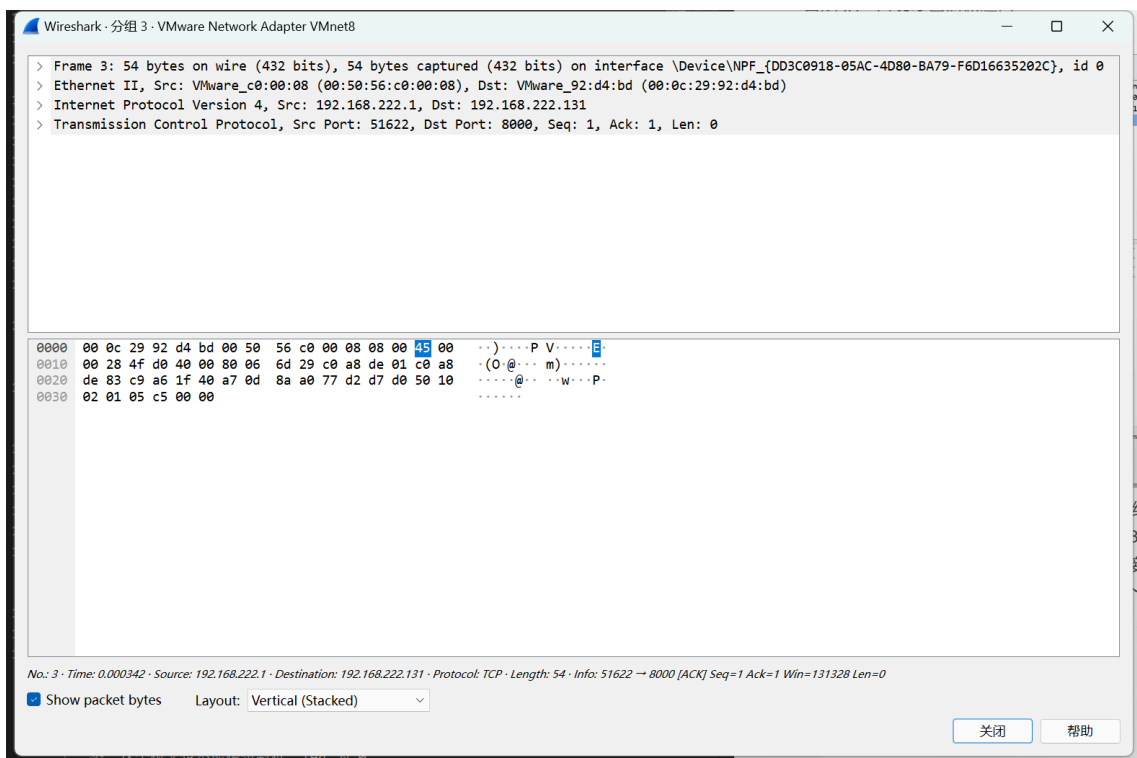
在输入了网址之后，客户端要主动和服务端建立连接，因此第一次握手是客户端向服务器发送了 TCP 请求。在这个数据包内容中我们可以看到客户端和服务器的源 IP 和目的 IP，以及各自对应端口。

第二次握手：



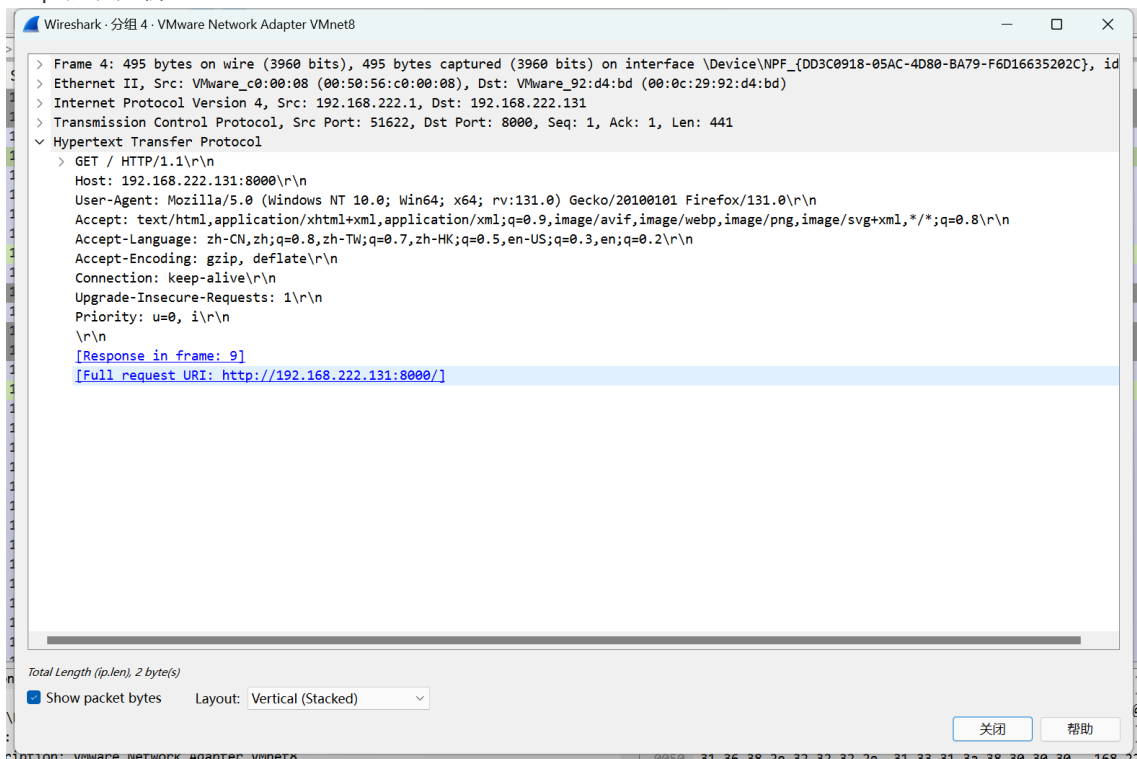
服务器接收到了客户端发起的连接请求后，给浏览器的应答，并在之后为该 TCP 分配缓存和变量。可见这条 TCP 报文是服务器 8080 端口发向主机 51622 端口的。服务器收到客户端的 SYN 报文段，如图中同意连接，则发出确认报文，确认报文中 Ack=1，同时，自身的相对序号 Seq 也会随机出一个值，这次为 0。服务器端将上述所有信息放到一个报文段发送给客户端，此时，TCP 服务器进程进入同步收到状态。这个报文也不能携带数据，len 为 0。

第三次握手：



客户端收到服务器回应的确认报文后，给服务器的应答，并为 TCP 连接分配缓存和变量。这里的 flag 中只有 Ack 一位有效，表示这是一条确认报文。报文序号为第二次握手手中的报文序号 +1。经过三次握手之后，客户端和服务端都进入了 ESTABLISHED 状态，完成 TCP 三次握手。双方确认连接完成，可以进行数据的传输交互。

## 2. http 协议分析：



这个数据包是三次握手之后传输的第一个 http 数据包。是 http 的请求报文，即tcp握手成功后，通过tcp/ip通信协议将客户端生成的 http 报文，发送至服务器。

这里我们看这个数据包是从 192.168.222.1 发送到 192.168.222.131，即是客户端向服务器发送的数据包。

这里我们详细查看数据包内容：

**请求行：**

```
✓ GET / HTTP/1.1\r\n
Request Method: GET
Request URI: /
Request Version: HTTP/1.1
```

表明为 Get 请求，http1.1 协议版本。

请求头：

```
Host: 192.168.222.131:8000\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:131.0) Gecko/20100101 Firefox/131.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8\r\n
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2\r\n
Accept-Encoding: gzip, deflate\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
Priority: u=0, i\r\n
\r\n
```

**Host:192.168.222.131:8000\r\n**：声明连接的目标主机和端口号。

**User-Agent**：这个字段提供了关于发起请求的浏览器和操作系统的信息，**Mozilla/5.0** 是浏览器的标识；**Windows NT 10.0; Win64; x64** 表示操作系统是 Windows 10 64 位；**Firefox/131.0** 表示这是火狐浏览器 131.0 版本。

**Accept**：这表示客户端可以识别的响应内容。**text/html**: HTML 文档；

**application/xhtml+xml**: XHTML 文档；**application/xml;q=0.9**：XML 格式的应用类型数据，“q=0.9”通常是质量因子的表示，说明对于这种媒体类型的偏好程度为 0.9；**image/avif**: AVIF 格式的图片；**image/webp**: WEBP 格式的图片；**image/png**: PNG 格式的图片；**\*/\*;q=0.8**: 其他任何类型的内容，但质量系数是 0.8。

**Accept-Language**：客户端接受的语言。

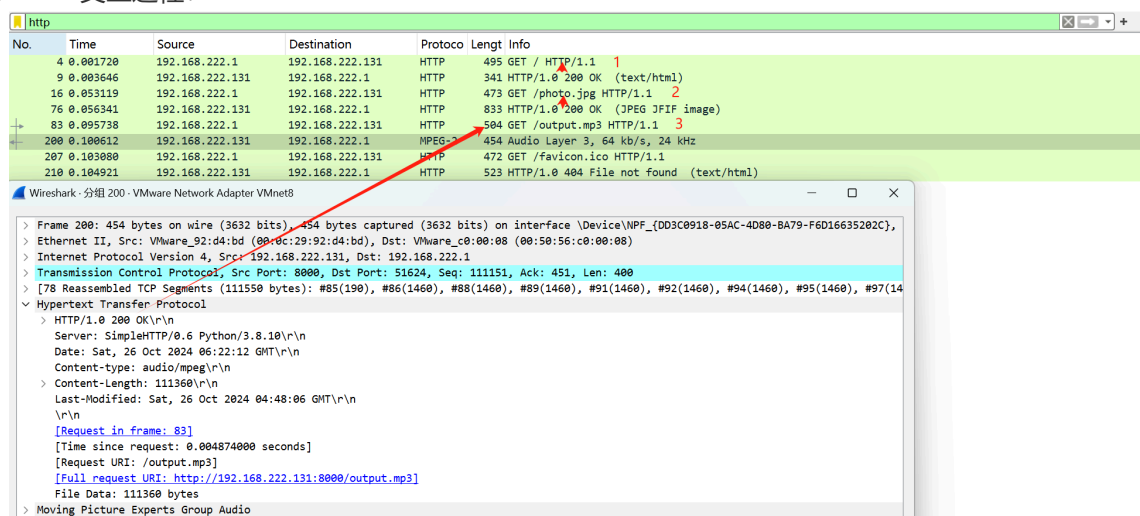
**Accept-Encoding**：客户端接受的编码形式。

**Connection**：对于 http 连接的处理，keep-alive 表示保持连接，代表这是一个长连接，这也是 http1.1 协议默认的连接方式。

**Upgrade-Insecure-Requests**：表示浏览器想要升级为安全连接。

**\r\n**：空行。

### 3. HTTP 交互过程：



The image shows a Wireshark network capture of an HTTP interaction. The packet list at the top shows three GET requests from 192.168.222.1 to 192.168.222.131:8000. The first request is for index.html, the second for photo.jpg, and the third for output.mp3. All three requests receive a 200 OK response. The packet details for the first request (index.html) are expanded, showing the HTTP 1.0 response structure. The response is from SimpleHTTP/0.6 Python/3.8.10, dated Sat, 26 Oct 2024 06:22:12 GMT. The content type is audio/mpeg, and the content length is 111360 bytes. The last modified time is Sat, 26 Oct 2024 04:48:06 GMT. The response is a 200 OK status.

根据筛选过后的数据包，我们可以发现在连接之后，客户端分别向服务器发送了三次 Get 请求，分别是针对 index.html 文档、LOGO 图片和介绍音频，并返回了三个 200。

断开连接（四次挥手）：

|              |                 |                 |        |                                                                                            |
|--------------|-----------------|-----------------|--------|--------------------------------------------------------------------------------------------|
| 199 0.100601 | 192.168.222.131 | 192.168.222.1   | TCP    | 1514 8000 → 51624 [ACK] Seq=109691 Ack=451 Win=64128 Len=1460 [TCP PDU reassembled in 200] |
| 200 0.100612 | 192.168.222.131 | 192.168.222.1   | MPEG-2 | 454 Audio Layer 3, 64 kb/s, 24 kHz                                                         |
| 201 0.100638 | 192.168.222.1   | 192.168.222.131 | TCP    | 54 51624 → 8000 [ACK] Seq=451 Ack=111552 Win=1049600 Len=0                                 |
| 202 0.100791 | 192.168.222.1   | 192.168.222.131 | TCP    | 54 51624 → 8000 [FIN, ACK] Seq=451 Ack=111552 Win=1049600 Len=0                            |
| 203 0.100987 | 192.168.222.131 | 192.168.222.1   | TCP    | 60 8000 → 51624 [ACK] Seq=111552 Ack=452 Win=64128 Len=0                                   |
| 204 0.102457 | 192.168.222.1   | 192.168.222.131 | TCP    | 66 51625 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM                      |
| 205 0.102796 | 192.168.222.131 | 192.168.222.1   | TCP    | 66 8000 → 51625 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128           |
| 206 0.102867 | 192.168.222.1   | 192.168.222.131 | TCP    | 54 51625 → 8000 [ACK] Seq=1 Ack=1 Win=1049600 Len=0                                        |
| 207 0.103080 | 192.168.222.1   | 192.168.222.131 | HTTP   | 472 GET /favicon.ico HTTP/1.1                                                              |
| 208 0.103302 | 192.168.222.131 | 192.168.222.1   | TCP    | 60 8000 → 51625 [ACK] Seq=1 Ack=419 Win=64128 Len=0                                        |
| 209 0.104732 | 192.168.222.131 | 192.168.222.1   | TCP    | 239 8000 → 51625 [PSH, ACK] Seq=1 Ack=419 Win=64128 Len=185 [TCP PDU reassembled in 210]   |
| 210 0.104921 | 192.168.222.131 | 192.168.222.1   | HTTP   | 523 HTTP/1.0 404 File not found (text/html)                                                |
| 211 0.104966 | 192.168.222.1   | 192.168.222.131 | TCP    | 54 51625 → 8000 [ACK] Seq=419 Ack=656 Win=1048832 Len=0                                    |
| 212 0.105104 | 192.168.222.1   | 192.168.222.131 | TCP    | 54 51625 → 8000 [FIN, ACK] Seq=419 Ack=656 Win=1048832 Len=0                               |
| 213 0.105436 | 192.168.222.131 | 192.168.222.1   | TCP    | 60 8000 → 51625 [ACK] Seq=656 Ack=420 Win=64128 Len=0                                      |
| 214 4.588306 | 192.168.222.1   | 239.255.255.250 | SSDP   | 209 M-SEARCH * HTTP/1.1                                                                    |
| 215 5.588902 | 192.168.222.1   | 239.255.255.250 | SSDP   | 209 M-SEARCH * HTTP/1.1                                                                    |
| 216 6.594765 | 192.168.222.1   | 239.255.255.250 | SSDP   | 209 M-SEARCH * HTTP/1.1                                                                    |
| 217 7.598382 | 192.168.222.1   | 239.255.255.250 | SSDP   | 209 M-SEARCH * HTTP/1.1                                                                    |

由于TCP连接是全双工的，因此每个方向都必须单独进行关闭。

当一方完成它的数据发送任务后就能发送一个 FIN 来终止这个方向的连接。收到一个 FIN 只意味着这一方向上没有数据流动，一个 TCP 连接在收到一个 FIN 后仍能发送数据。首先进行关闭的一方将执行主动关闭，而另一方执行被动关闭。大致分为以下四步：

- 1.服务器读通道关闭
- 2.客户机写通道关闭
- 3.客户机读通道关闭
- 4.服务器写通道关闭

关闭行为是在发起方数据发送完毕之后，给对方发出一个 FIN（finish）数据段。直到接收到对方发送的 FIN，且对方收到了接收确认 ACK 之后，双方的数据通信完全结束，过程中每次接收都需要返回确认数据段 ACK。